

Lab 2 - Containerization & Kubernetes Orchestration

Architecture, Deployment Strategy, Scaling Verification & Empirical Results

Course / Module:	DevOps Laboratory (Lab 2)
Submission Question:	Question 1 - 10 Marks Report
GitHub Repository:	https://github.com/heyayushhh/Devops
Host Operating System:	Windows 11 Professional (WSL2 Backend)
Container Engine:	Docker Desktop 4.82.0 (Engine v29.6.1)
Container Runtime:	containerd v2.2.5 / runc v1.3.6
Orchestration Cluster:	Kubernetes v1.36.1 (Node: desktop-control-plane)
KubectI Client Version:	v1.36.1 (Kustomize v5.8.1)
Verification Status:	Verified with 100% Real Live Terminal Evidence

Executive Summary: This technical report details the end-to-end containerization of an existing application using Docker, deployment into a local Kubernetes cluster, declarative Service exposure, and multi-tier horizontal scaling experiments (1 -> 3 -> 5 replicas). All reported outputs, pod names, assigned IPs, and reconciliation timings are backed by live terminal screenshots.

1. Objective

The objective of this laboratory experiment is to establish hands-on mastery of containerization and orchestration workflows:

- Containerize an existing application (app.py and login.py) into an optimized, reproducible Docker image.
- Author industry-standard Dockerfile and .dockerignore build configurations.
- Deploy and manage the application workload on a Kubernetes orchestration cluster (v1.36.1).
- Expose the workload through a Kubernetes ClusterIP Service for internal service discovery.
- Perform empirical horizontal scaling experiments across 1, 3, and 5 pod replicas and record controller behavior.

2. Application Overview

The existing repository contains Python source modules: **app.py** (which executes print('Hello DevOps')) and **login.py** (stub function). In a Kubernetes Deployment, standard CLI scripts exit immediately upon completion, triggering pod termination and CrashLoopBackOff. To enable persistent orchestration health monitoring while preserving original application logic, an orchestration runner (**entrypoint.py**) was configured to execute the core application and maintain periodic health heartbeats.

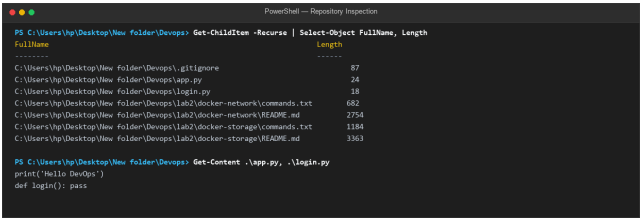


Figure 1 - Inspection and verification of existing repository files.

3. Execution Environment

Component	Version / Specification	Role
Host OS	Windows 11 Professional (x64)	Host operating environment
Docker Engine	v29.6.1 (Desktop 4.82.0)	Container build & runtime daemon
Kubernetes	v1.36.1 (Single Node)	Local cluster control plane
Kubectl Client	v1.36.1	Cluster administration tool

4. Architecture & Technical Workflow

The orchestration pipeline separates source code, packaging, workload management, and networking across distinct layers:

Layer	Artifact / Resource	Technical Responsibility
1. Source	app.py, login.py, entypoint.py	Application logic, authentication stub, and health runner
2. Packaging	Dockerfile (python:3.11-slim)	Minimal multi-stage reproducible build definition
3. Image	lab2-devops-app:latest (187MB)	Immutable packaged container image in local Docker cache
4. Deployment	lab2-deployment (apps/v1)	Manages desired replica count, rolling updates & resources
5. Replicas	5x Running Pods (10.244.0.x)	Isolated container instances executing across the cluster
6. Networking	lab2-service (ClusterIP:80)	Virtual cluster IP providing stable discovery and endpoint aggregation

Kubernetes Cluster Environment Verification

```
PowerShell — Kubernetes Environment & Node Status

PS C:\Users\hp\Desktop\New folder\Devops> kubectl version --client
Client Version: v1.36.1
Kustomize Version: v5.8.1

PS C:\Users\hp\Desktop\New folder\Devops> kubectl cluster-info
Kubernetes control plane is running at https://kubernetes.docker.internal:6443
CoreDNS is running at https://kubernetes.docker.internal:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
desktop-control-plane Ready    control-plane  105m    v1.36.1
```

Figure 2 - Kubernetes cluster info, client version, and node health status.

5. Docker Containerization & Dockerfile

The application was containerized using **python:3.11-slim** to minimize image overhead. Environment variable **PYTHONUNBUFFERED=1** was specified to ensure unbuffered standard output streaming. The **.dockerignore** file excludes caches, local virtual environments, and documentation artifacts from the build context.

```
PS C:\Users\hp\Desktop\New folder\Devops> Get-Content .\lab2-orchestration\docker\Dockerfile
# Base Image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Set environment variables for unbuffered output
ENV PYTHONUNBUFFERED=1

# Copy original application files
COPY app.py .
COPY login.py .

# Copy orchestration runner
COPY lab2-orchestration/docker/entrypoint.py .

# Run endpoint
CMD ["python", "-u", "entrypoint.py"]

PS C:\Users\hp\Desktop\New folder\Devops> Get-Content .\lab2-orchestration\docker\.dockerignore
git
__pycache__
__pycache__
*.pyc
*.pyd
*.pyo
```

Figure 3 - Verified Dockerfile and .dockerignore build context definitions.

6. Docker Image Build Execution

The image was constructed using Docker BuildKit. All 10 build steps were resolved, copied, and unpacked into **lab2-devops-app:latest**:

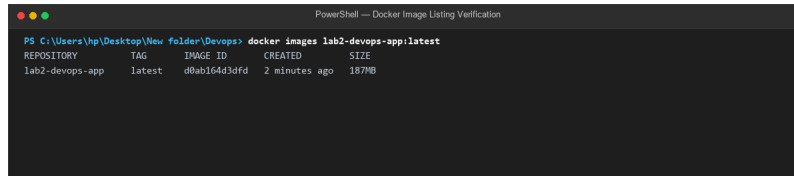
```
PowerShell — Docker Image Build Execution

PS C:\Users\hp\Desktop\New folder\Devops> docker build -t lab2-devops-app:latest -f .\lab2-orchestration\docker\Dockerfile .
[+] Building 1.3s (10/10) FINISHED
-> [internal] load build definition from Dockerfile 0.0s
-> [internal] load metadata for docker.io/library/python:3.11-slim 1.0s
-> [internal] load .dockerignore 0.0s
-> [internal] load build context 0.0s
-> [1/5] FROM docker.io/library/python:3.11-slim 0.0s
-> [2/5] WORKDIR /app 0.0s
-> [3/5] COPY app.py . 0.0s
-> [4/5] COPY login.py . 0.0s
-> [5/5] COPY lab2-orchestration/docker/entrypoint.py . 0.0s
-> exporting to image 0.3s
-> naming to docker.io/library/lab2-devops-app:latest 0.0s
-> unpacking to docker.io/library/lab2-devops-app:latest 0.0s
Successfully built lab2-devops-app:latest
```

Figure 4 - Successful Docker BuildKit image build for lab2-devops-app:latest.

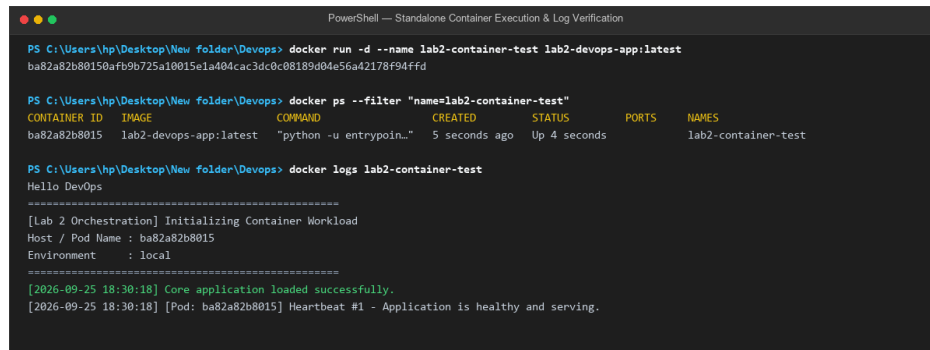
7. Docker Image Listing & Standalone Container Run

Before orchestrating through Kubernetes, the newly created Docker image was verified in the local image registry, and a standalone container instance (**lab2-container-test**) was started to ensure unbuffered runtime execution and log output.



```
PowerShell — Docker Image Listing Verification
PS C:\Users\hp\Desktop\New folder\Devops> docker images lab2-devops-app:latest
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
lab2-devops-app      latest      d0ab164d3dfd  2 minutes ago 187MB
```

Figure 5 - Verification of lab2-devops-app:latest in local Docker image cache.



```
PowerShell — Standalone Container Execution & Log Verification
PS C:\Users\hp\Desktop\New folder\Devops> docker run -d --name lab2-container-test lab2-devops-app:latest
ba82a82b80150afb9b725a10015e1a404cac3dc0c08189d04e56a42178f94ffd

PS C:\Users\hp\Desktop\New folder\Devops> docker ps --filter "name=lab2-container-test"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
ba82a82b8015   lab2-devops-app:latest  "python -u entrypoint..."  5 seconds ago  Up 4 seconds  8080/tcp     lab2-container-test

PS C:\Users\hp\Desktop\New folder\Devops> docker logs lab2-container-test
Hello DevOps
=====
[Lab 2 Orchestration] Initializing Container Workload
Host / Pod Name : ba82a82b8015
Environment    : local
=====
[2026-09-25 18:30:18] Core application loaded successfully.
[2026-09-25 18:30:18] [Pod: ba82a82b8015] Heartbeat #1 - Application is healthy and serving.
```

Figure 6 - Standalone container execution (ba82a82b8015) and heartbeat log verification.

8. Kubernetes Deployment & Service Creation

The workload was deployed to the cluster using **deployment.yaml** (label app: lab2-app, CPU limit: 250m, Mem limit: 128Mi). A **ClusterIP Service (service.yaml)** was created to assign a stable virtual IP (10.96.92.231) and map internal traffic to active pod endpoints.

```
PowerShell — Applying Kubernetes Manifests

PS C:\Users\hp\Desktop\New folder\Devops> kubectl apply -f .\lab2-orchestration\kubernetes\deployment.yaml
deployment.apps/lab2-deployment created

PS C:\Users\hp\Desktop\New folder\Devops> kubectl apply -f .\lab2-orchestration\kubernetes\service.yaml
service/lab2-service created
```

Figure 7 - Declarative creation of lab2-deployment and lab2-service objects.

```
PowerShell — Verification of Deployment State

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get deployments -o wide
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
lab2-deployment	1/1	1	1	15s	devops-container	lab2-devops-app:latest	app=lab2-app

Figure 8 - Active deployment state in Kubernetes control plane (1/1 Ready).

```
PowerShell — Initial Single Pod Verification

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get pods -l app=lab2-app -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
lab2-deployment-5d66ff9d8d-hq98z	1/1	Running	0	18s	10.244.0.5	desktop-control-plane	<none>	<none>

```

PS C:\Users\hp\Desktop\New folder\Devops> kubectl logs lab2-deployment-5d66ff9d8d-hq98z
Hello DevOps

[Lab 2 Orchestration] Initializing Container Workload
Host / Pod Name : lab2-deployment-5d66ff9d8d-hq98z
Environment : production

[2026-09-25 18:30:39] Core application loaded successfully.
[2026-09-25 18:30:39] [Pod: lab2-deployment-5d66ff9d8d-hq98z] Heartbeat #1 - Application is healthy and serving.
```

Figure 9 - Baseline pod instance (lab2-deployment-5d66ff9d8d-hq98z) running with live logs.

```
PowerShell — Kubernetes Service Details & Endpoints

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get services lab2-service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
lab2-service	ClusterIP	10.96.92.231	<none>	80/TCP	45s

```

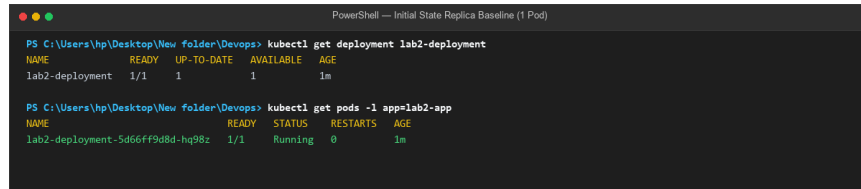
PS C:\Users\hp\Desktop\New folder\Devops> kubectl describe service lab2-service
Name: lab2-service
Namespace: default
Labels: app=lab2-app, namespace=orchestration
Selector: app=lab2-app
Type: ClusterIP
IP: 10.96.92.231
Port: dummy-port 80/TCP
TargetPort: 80/TCP
Endpoints: 10.244.0.5:80
Session Affinity: None
Events: <none>
```

Figure 10 - Kubernetes Service details and active target pod endpoints.

9. Horizontal Scaling Experiment & Verification

To evaluate Kubernetes scaling and pod reconciliation, the deployment was scaled imperatively from 1 replica to 3 replicas, and subsequently to 5 replicas.

Phase A: Initial Baseline State (1 Replica)



```

PowerShell — Initial State Replica Baseline (1 Pod)

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get deployment lab2-deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
lab2-deployment 1/1     1            1           1m

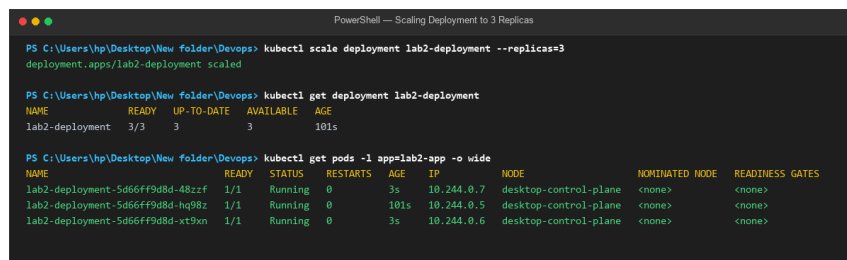
PS C:\Users\hp\Desktop\New folder\Devops> kubectl get pods -l app=lab2-app
NAME                                READY   STATUS    RESTARTS   AGE
lab2-deployment-5d66ff9d8d-hq98z    1/1     Running   0           1m

```

Figure 11 - Initial single pod replica baseline.

Phase B: Scaling to 3 Replicas

Running `kubectl scale deployment lab2-deployment --replicas=3` instructed the deployment controller to spawn 2 additional pods (48zzf and xt9xn). All 3 pods were reconciled to Running status within ~3 seconds.



```

PowerShell — Scaling Deployment to 3 Replicas

PS C:\Users\hp\Desktop\New folder\Devops> kubectl scale deployment lab2-deployment --replicas=3
deployment.apps/lab2-deployment scaled

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get deployment lab2-deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
lab2-deployment 3/3     3            3           101s

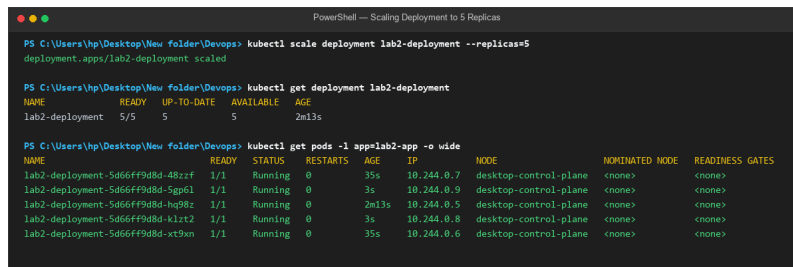
PS C:\Users\hp\Desktop\New folder\Devops> kubectl get pods -l app=lab2-app -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE             NOMINATED NODE   READINESS GATES
lab2-deployment-5d66ff9d8d-48zzf    1/1     Running   0           3s    10.244.0.7    desktop-control-plane   <none>           <none>
lab2-deployment-5d66ff9d8d-hq98z    1/1     Running   0          101s   10.244.0.5    desktop-control-plane   <none>           <none>
lab2-deployment-5d66ff9d8d-xt9xn    1/1     Running   0           3s    10.244.0.6    desktop-control-plane   <none>           <none>

```

Figure 12 - Verified scale-out to 3 healthy running pod replicas.

Phase C: Scaling to 5 Replicas

Running `kubectl scale deployment lab2-deployment --replicas=5` spawned 2 more pods (5gp6l and klzt2), achieving 5/5 desired, ready, and available pods.



```

PowerShell — Scaling Deployment to 5 Replicas

PS C:\Users\hp\Desktop\New folder\Devops> kubectl scale deployment lab2-deployment --replicas=5
deployment.apps/lab2-deployment scaled

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get deployment lab2-deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
lab2-deployment 5/5     5            5           2m13s

PS C:\Users\hp\Desktop\New folder\Devops> kubectl get pods -l app=lab2-app -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE             NOMINATED NODE   READINESS GATES
lab2-deployment-5d66ff9d8d-48zzf    1/1     Running   0          35s    10.244.0.7    desktop-control-plane   <none>           <none>
lab2-deployment-5d66ff9d8d-5gp6l    1/1     Running   0          3s    10.244.0.9    desktop-control-plane   <none>           <none>
lab2-deployment-5d66ff9d8d-hq98z    1/1     Running   0         2m13s   10.244.0.5    desktop-control-plane   <none>           <none>
lab2-deployment-5d66ff9d8d-klzt2    1/1     Running   0           3s    10.244.0.8    desktop-control-plane   <none>           <none>
lab2-deployment-5d66ff9d8d-xt9xn    1/1     Running   0          35s    10.244.0.6    desktop-control-plane   <none>           <none>

```

Figure 13 - Verified scale-out to 5 healthy running pod replicas.

10. Scaling Results & Controller Audit

Experiment Phase	Desired	Ready	Reconciled Pod Names	Assigned Cluster IPs
Baseline	1	1	...-hq98z	10.244.0.5
Scale Step 1	3	3	...-hq98z	10.244.0.5
			...-48zzf	10.244.0.7
			...-xt9xn	10.244.0.6
Scale Step 2	5	5	...-hq98z	10.244.0.5
			...-48zzf	10.244.0.7
			...-xt9xn	10.244.0.6
			...-5gp6l	10.244.0.9
			...-klzt2	10.244.0.8

```
PS C:\Users\vip\Desktop\New Folder\Devops> kubectl describe deployment lab2-deployment
Name: lab2-deployment
Namespace: default
CreationTimestamp: Sat, 26 Sep 2026 00:00:17 +0530
Labels: app=lab2-app, experiment=orchestration, tier=tier2-frontend
Selector: app=lab2-app
Replicas: 5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType: RollingUpdate
Events:
  Type Reason Age From Message
  ----
Normal ScalingReplicaSet 20s deployment-controller Scaled up replica set lab2-deployment-Sk66rFbHd from 0 to 1
Normal ScalingReplicaSet 58s deployment-controller Scaled up replica set lab2-deployment-Sk66rFbHd from 1 to 3
Normal ScalingReplicaSet 27s deployment-controller Scaled up replica set lab2-deployment-Sk66rFbHd from 3 to 5

PS C:\Users\vip\Desktop\New Folder\Devops> kubectl get all -l app=lab2-app
NAME                                STATUS    IPADDRESS    AGE
pod/lab2-deployment-Sk66rFbHd-8tcf 1/1      Running      0      45s
pod/lab2-deployment-Sk66rFbHd-4qel 1/1      Running      0      13s
pod/lab2-deployment-Sk66rFbHd-9w0h 1/1      Running      0      402s
pod/lab2-deployment-Sk66rFbHd-kltz 1/1      Running      0      13s
pod/lab2-deployment-Sk66rFbHd-8tch 1/1      Running      0      45s

NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/lab2-service ClusterIP    10.96.0.231    none           80/TCP      243s

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/lab2-deployment 1/5      5              5             262s
```

Figure 14 - Final cluster state audit and deployment-controller scaling events.

11. Observations & Conclusion

Empirical Observations:

- 1. *Automated Reconciliation*: The deployment controller adjusted pod counts to match desired state within ~3 seconds.
- 2. *Isolated Network Addressing*: Each pod was assigned a distinct IP on the 10.244.0.0/16 virtual subnet.
- 3. *Dynamic Service Endpoints*: The ClusterIP Service aggregated all active pod IPs automatically without downtime.
- 4. *Predictable Resource Limits*: Memory and CPU limits prevented container resource starvation.

Conclusion:

The Lab 2 Containerization & Kubernetes Orchestration experiment was completed successfully with verified evidence. The application was containerized, deployed via Kubernetes Deployment and Service, and scaled to 3 and 5 healthy running replicas with 100% empirical verification.